Home Exam 2

IN4230 Nettverk

Candidate number

15169

Connection synchronizing

If a protocol is unsynchronized, this can lead to overlapping problems between two endpoints. Both endpoints need to agree on which data belongs to which connection, and maintain a consistent connection state. Without synchronization, data from a previous session might "bleed" into a new one, causing packets to be misplaced, mixed, or unnecessarily discarded.

The role of connection synchronization is therefore to ensure that both endpoints share the same understanding of the connection state, so that data integrity is preserved throughout a transfer.

In my implementation, when a new connection is initiated from an application, each MIPTP daemon stores the connection in a data structure containing a list of active application transfers. When a message is received from the MIP daemon, the MIPTP daemon checks the header for the source MIP and source port. If this combination matches an existing active transfer, the message is routed to the corresponding destination. If not, a control message is sent up to the destination application, notifying it that a new connection is being established so it can create a new transfer structure. Incoming messages are then matched against this table of active transfers using the (src MIP, src port) pair, ensuring that data is written to the correct destination and maintaining synchronization between endpoints.

Another possible solution is to introduce a three-way handshake, similar to TCP (Postel, 1981). The client could send a SYN message with an initial sequence number, the server would respond with a SYN-ACK, and the client would complete the handshake with an ACK. This approach increases complexity, but it ensures both sides have a shared and synchronized view of sequence numbers and connection state before any data is transferred.

Alternatively, one could use a session ID instead of relying solely on the (src MIP, src port) pair to identify a transfer, like the QUIC transport protocol does (J. Iyengar & M. Thomson, 2021). This would reduce the risk of overlap if a new session happens to reuse the same port and address combination. A randomly generated session ID would make it highly unlikely that a new session could be mistaken for an old one.

Retransmission timeout value

In this implementation, the retransmission timeout value is statically set to 0.2 seconds and does not adapt to network conditions. In a real transport protocol, this would not be a robust approach since network conditions vary over time. For example, the round-trip time (RTT) can increase due to congestion or routing delays. If the timeout is shorter than the actual RTT, the sender will incorrectly assume that a packet is lost, leading to unnecessary retransmissions and reduced efficiency. On the other hand, if the timeout is set too high, the sender will wait too long before detecting a loss, which slows down recovery and decreases overall performance (Welzl, 2025b). This is why it is necessary to continuously adjust the timeout value based on the measured RTT.

Since RTT varies you have to know how fast to react to it, so that the average is representative of the actual network conditions. One approach is to use an Exponentially Weighted Moving Average to smooth out short-term fluctuations in RTT measurements. The estimate reacts to changes depending on the weight factor α, and the final timeout is calculated by adding a weighted measure of the RTT variation to this estimate (Welzl, 2025b). In my MIPTPD implementation, this could be added by continuously measuring RTTs from acknowledgments and updating the timeout dynamically based on the smoothed estimate.

The problem with this approach is that it can be unclear whether an acknowledgment refers to the original or a retransmitted segment, which makes it difficult to determine when to stop the timer accurately. An approach to handle this is described by the Karn/Partridge algorithm, where the RTT samples from retransmitted packets simply are ignored to avoid this ambiguity (Welzl, 2025b).

Alternatively, a timestamp option can be included in the transport header. This allows the sender to record the sending time directly in the packet, and the receiver to reflect this value in its acknowledgment, ensuring that each RTT measurement can be accurately matched to the correct segment (Welzl, 2025b). To support this in my MIPTPD, the protocol could include a timestamp field in the header, making RTT calculations precise even when retransmissions occur.

Additional mechanisms

A transport layer can include several mechanisms to ensure reliable, efficient, and orderly communication between endpoints. Beyond basic reliability, additional mechanisms such as error control, flow control, congestion control, and connection management contribute to maintaining stable communication under varying network conditions (Welzl, 2025a, 2025b).

In my MIPTP daemon, the Go-Back-N strategy is used to provide reliability. In this approach, if one packet within the window is lost or times out, the sender retransmits all packets from the last acknowledged one onward (Welzl, 2025b). A more efficient extension would be the Selective Repeat strategy, where each packet is acknowledged individually, and only the missing ones are retransmitted. This requires more complex buffering and state management on both the sender and receiver, but significantly improves performance when packets are lost or arrive out of order (Weldon, 1982).

Flow control

Another useful mechanism to include in the transport layer is flow control. Flow control is used to limit how much data can be sent before it is acknowledged, in order to protect the receiver from getting more data than they are able to process. This is represented by a receiver window (Welzl, 2025a). Flow control could be implemented in my MIPTPD by allowing the receiver to advertise its available buffer space in each ACK message. The sender would then be able to limit the number of unacknowledged packets it sends based on this, pausing transmission when the receiver's buffer is full and starting transmission once new ACKs indicate that space is available.

Congestion control

While flow control protects the receiver, congestion control protects the network itself from overload by limiting the sending rate. Congestion control is implemented by reducing and increasing the sending rate based on signals of delay or loss. The sender has a congestion window, which tells how many packets that can be in transit without acknowledgment (Welzl, 2025a). In my implementation the cwnd is set as a static value, but the transport layer should have this as a dynamic variable. A fixed window cannot react to changing network conditions,

and therefore neither backs off during congestion or increases throughput when the network is less loaded.

A dynamic congestion control mechanism could follow the principles of TCP Tahoe (Welzl, 2025a). The sender starts with a small window and increases it exponentially with each ACK received (slow start), then switches to linear growth (congestion avoidance). When a timeout or packet loss occurs, it is interpreted as a sign of congestion, and the sender reduces its sending rate using multiplicative decrease.

In modern high-speed networks, however, the capacity is much higher, and the required congestion window must be very large to fully utilize the link. Therefore, the congestion control mechanism needs to increase cwnd faster and react less aggressively to occasional losses, as the recovery from a large decrease would otherwise take a very long time (Welzl, 2025a). In the context of this assignment, where MIPTPD operates on a single local machine, such mechanisms are less critical, as bandwidth and delay are stable.

Connection management

Finally, connection management is a natural part of a transport protocol, and define the ways a connection is established, maintained and finished end to end in a controlled way, for example as demonstrated in RFC 793 (Postel, 1981). An example is graceful connection termination, which prevents data loss or misinterpretation of delayed packets. For example, similar to TCP, a closing side could first signal that it has no more data to send (a “CLOSE” operation) while still accepting incoming data until the peer also closes. This ensures that all sent data is reliably delivered before the connection is fully terminated, allowing both sides to shut down in a controlled manner (Postel, 1981).

In my MIPTPD implementation, when an application socket closes, the MIPTP daemon checks for unacknowledged packets before removing the connection entry. The daemon waits briefly to allow delayed ACKs to arrive, ensuring that the connection is closed safely and that no data is lost. In contrast to protocols with coordinated connection teardown, my implementation cleans up state locally and independently on each side. The client removes its transfer state when its application closes, while the server continues running and only frees the specific transfer entry once all expected data has been received.

Notes on use of AI

I used AI as a support tool to help me understand C programming as I did not have previous experience with the language before taking this course. I also used it to debug errors, and improve the structure of my code. In some cases, I also received suggestions for code snippets, which I adapted and integrated myself after verifying them against the assignment requirements. Additionally, I used AI to help me reason about how to design a Mininet test script, so that I could validate my own implementation.

Bibliography

J. Iyengar, Ed., & M. Thomson, E. (2021, May). *RFC 9000 QUIC: A UDP-Based Multiplexed and Secure Transport*. Internet Engineering Task Force (IETF). https://www.rfc-editor.org/rfc/rfc9000

Postel, J. (1981). *Transmission Control Protocol – DARPA Internet Program Protocol Specification (RFC 793)*. Internet Engineering Task Force (IETF). https://www.rfc-editor.org/rfc/rfc793.html

Weldon, E. J., Jr. (1982, March 31). *An Improved Selective-Repeat ARQ Strategy*. IEEE Transactions on Communications. 10.1109/TCOM.1982.1095497

Welzl, M. (2025a, August 28). *Congestion control* [Lecture]. https://www.uio.no/studier/emner/matnat/ifi/IN3230/h25/kursmateriell/in3230_cc.pdf

Welzl, M. (2025b, September 18). *The Internet transport layer* [Lecture]. https://whttps//www.uio.no/studier/emner/matnat/ifi/IN3230/h25/kursmateriell/in3230_transportlayer.pdf

MIPTPD

CLIENT

Open file and find file size

Connect to MIPTPD through UNIX socket

connection successful?

close app and file

Register random port number tries up to three times

port successfully registered

Write first message: dest info + file size

read file and divide in 1400 byte data chunks

put together packet with dst_mip and dst_port given through command parameters

send data too MIPTPD

MIPTPD

SERVER

Connect to MIPTPD through UNIX socket

connection successful?

close app and file

register port number

go into receiving loop

leser melding fra MIPTPD

Control message?

Yes

create new transfer

No

Find transfer in transfer list

File size message?

Yes

sett expected size

No

Write payload to file

received > expected size?

Close file and transfer

Keep receiving data

MIPTPD

Connect to MIPD through UNIC socket

Register SDU type with MIPD

Create unix socket for listening for

start event loop

Event received

Which event?

Data from MIPD

New app connection

Message from existing app

Read packet

ACK OR data

ACK

Data

Update base seq and free window

Empty queue if space in window

ny (src mip, src port?

Register new transfer

Tell Server that new transfer is starting

Control seq number and If in order

Deliver in order packet

Send ACK for last in order packet

Register app and port

Put app in epoll for monitoring

Read destination of packet

Remove metadata

WIndow full?

Yes

Put packet in queue

No

Set sequence number and build MIPTP PDU

Put in sender window and send to MIPD